

```

import os
import json
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def build_prompt(metrics: list[dict]) -> str:
    """
    Turns raw metric rows into a compact, structured prompt.
    We keep this deterministic and simple: the model's job is
    to summarize numbers we already computed, NOT to do math itself
    (LLMs are unreliable at arithmetic -- so any aggregation, like
    percentage change, should be computed in Python first and just
    described in the prompt).
    """
    lines = [f"- {m['metric_name']}: {m['metric_value']} (recorded {m['recorded_a']}"
             for m in metrics]
    metrics_block = "\n".join(lines)

    return f"""You are a business analyst. Given the following KPI data points
    for a company, write a concise 3-4 sentence executive summary highlighting
    notable trends, any concerning drops, and one actionable recommendation.
    Do not invent numbers that are not in the data.

    KPI DATA:
    {metrics_block}

    Respond in plain text, no markdown."""

def generate_kpi_summary(metrics: list[dict]) -> str:
    if not metrics:
        return "No metrics available yet for this tenant."

    prompt = build_prompt(metrics)

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.3, # low temperature: we want consistent, factual summaries
        max_tokens=200,
    )

```

```
return response.choices[0].message.content.strip()
```